

1. Check if a number is EVEN or ODD.

- **Logic:** A number is even if `num % 2 == 0`; otherwise, it is odd.

```
public class EvenOdd {  
    public static void main(String[] args) {  
        int num = 5;  
        if (num % 2 == 0) {  
            System.out.println(num + " is Even");  
        } else {  
            System.out.println(num + " is Odd");  
        }  
    }  
}
```

Flowchart: Input → Check `num % 2 == 0` → Output result (Even or Odd).

Time Complexity: $O(1)$

Space Complexity: $O(1)$

2. Find the Factorial of a given number.

- **Logic:** The factorial of `n` is `n * (n-1) * (n-2) * ... * 1`.

```
public class Factorial {  
    public static void main(String[] args) {  
        int num = 5;  
        int factorial = 1;  
        for (int i = 1; i <= num; i++) {  
            factorial *= i;  
        }  
        System.out.println("Factorial of " + num + " is " + factorial);  
    }  
}
```

- **Flowchart:** Start → Initialize factorial to 1 → Loop from 1 to `n` → Multiply each number → Output factorial.
- **Time Complexity:** $O(n)$
- **Space Complexity:** $O(1)$

3. Factorial using Recursion.

- **Logic:** `factorial(n) = n * factorial(n-1)` with base case `factorial(0) = 1`.

```
public class FactorialRecursion {
```

```

public static int factorial(int n) {
    if (n == 0) return 1;
    return n * factorial(n - 1);
}

public static void main(String[] args) {
    System.out.println("Factorial of 5 is " + factorial(5));
}
}

```

- Flowchart: Start → Call **factorial(n)** → Recursive calls → Base case → Output result.
- Time Complexity: $O(n)$
- Space Complexity: $O(n)$ (due to recursion stack)

4. Swap two numbers without using a third variable (Approach 1: Arithmetic).

- Logic: $a = a + b$, then $b = a - b$, and $a = a - b$.

```

public class SwapNumbers {
    public static void main(String[] args) {
        int a = 10, b = 20;

        a = a + b;
        b = a - b;
        a = a - b;

        System.out.println("a = " + a + ", b = " + b);
    }
}

```

- Flowchart: Input → $a = a + b$ → $b = a - b$ → $a = a - b$ → Output.
- Time Complexity: $O(1)$
- Space Complexity: $O(1)$

5. Swap two numbers without using a third variable (Approach 2: XOR).

- Logic: $a = a \oplus b$, then $b = a \oplus b$, and $a = a \oplus b$.

```
public class SwapNumbersXOR {  
  
    public static void main(String[] args) {  
  
        int a = 10, b = 20;  
  
        a = a ^ b;  
  
        b = a ^ b;  
  
        a = a ^ b;  
  
        System.out.println("a = " + a + ", b = " + b);  
  
    }  
}
```

- Flowchart: Input $\rightarrow a = a \oplus b \rightarrow b = a \oplus b \rightarrow a = a \oplus b \rightarrow$ Output.
- Time Complexity: $O(1)$
- Space Complexity: $O(1)$

6. Swap two numbers without using a third variable (Approach 3: Using multiplication and division).

- Logic: $a = a * b$, then $b = a / b$, and $a = a / b$.

```
public class SwapNumbersMulDiv {  
  
    public static void main(String[] args) {  
  
        int a = 10, b = 20;  
  
        a = a * b;  
  
        b = a / b;  
  
        a = a / b;  
  
        System.out.println("a = " + a + ", b = " + b);  
  
    }  
}
```

}

- Flowchart: Input $\rightarrow a = a * b \rightarrow b = a / b \rightarrow a = a / b \rightarrow$ Output.
- Time Complexity: $O(1)$
- Space Complexity: $O(1)$

7. Check if the number is Positive or Negative.

- Logic: If the number is greater than 0, it's positive; if less than 0, it's negative.
- Code:

java

CopyEdit

```
public class PositiveNegative {  
  
    public static void main(String[] args) {  
  
        int num = -5;  
  
        if (num > 0) {  
  
            System.out.println(num + " is Positive");  
  
        } else if (num < 0) {  
  
            System.out.println(num + " is Negative");  
  
        } else {  
  
            System.out.println(num + " is Zero");  
  
        }  
  
    }  
  
}
```

- Flowchart: Input $\rightarrow$ Check if $num > 0$, $num < 0$, or $num == 0 \rightarrow$ Output.
 - Time Complexity: $O(1)$
 - Space Complexity: $O(1)$
-

8. Check if the given number is a Leap Year.

- Logic: A year is a leap year if it is divisible by 4 but not divisible by 100, except if it is divisible by 400.

```
public class LeapYear {  
  
    public static void main(String[] args) {  
        int year = 2024;  
        if ((year % 4 == 0 && year % 100 != 0) || year % 400 == 0) {  
            System.out.println(year + " is a Leap Year");  
        } else {  
            System.out.println(year + " is not a Leap Year");  
        }  
    }  
}
```

- Flowchart: Input → Check if `year % 4 == 0 && year % 100 != 0` or `year % 400 == 0` → Output.
- Time Complexity: $O(1)$
- Space Complexity: $O(1)$

10. Print the digits of a given number.

- Logic: Convert the number into digits by dividing it by 10 iteratively.

```
public class PrintDigits {  
    public static void main(String[] args) {  
        int num = 12345;  
        while (num > 0) {  
            System.out.println(num % 10);  
            num = num / 10;  
        }  
    }  
}
```

- Flowchart: Input → Loop to extract last digit (`num % 10`) → Print digit → Divide number by 10 → Repeat.
- Time Complexity: $O(d)$, where d is the number of digits in the number.
- Space Complexity: $O(1)$

11. Print all the Factors of the Given Number.

- Logic: Iterate from 1 to `n` and check if `n % i == 0`.

```
public class Factors {
    public static void main(String[] args) {
        int num = 12;
        for (int i = 1; i <= num; i++) {
            if (num % i == 0) {
                System.out.println(i);
            }
        }
    }
}
```

- Flowchart: Input → Loop from 1 to `num` → Check if `num % i == 0` → Print `i`.
- Time Complexity: $O(n)$
- Space Complexity: $O(1)$

12. Find the sum of the digits of a given number.

- Logic: Extract each digit using modulo operation and add it.

```
public class SumOfDigits {
    public static void main(String[] args) {
        int num = 12345;
        int sum = 0;
        while (num > 0) {
            sum += num % 10;
            num = num / 10;
        }
        System.out.println("Sum of digits: " + sum);
    }
}
```

- Flowchart: Input → Loop to extract last digit → Add to sum → Divide number by 10 → Repeat.
- Time Complexity: $O(d)$
- Space Complexity: $O(1)$

13. Find the smallest of 3 numbers without using `<` or `>` symbols.

- Logic: Use the `Math.min()` function to find the smallest number.

```
public class SmallestNumber {
```

```

public static void main(String[] args) {
    int a = 5, b = 10, c = 3;
    int smallest = Math.min(Math.min(a, b), c);
    System.out.println("Smallest number: " + smallest);
}
}

```

- Flowchart: Input → Use `Math.min()` to find the smallest → Output.
- Time Complexity: $O(1)$
- Space Complexity: $O(1)$

14. Add two numbers without using arithmetic operators.

- Logic: Use bitwise operations: XOR for addition and AND for carry.

```

public class AddWithoutOperators {
    public static void main(String[] args) {
        int a = 5, b = 10;
        while (b != 0) {
            int carry = a & b;
            a = a ^ b;
            b = carry << 1;
        }
        System.out.println("Sum: " + a);
    }
}

```

- Flowchart: Input → Use XOR to add → Calculate carry using AND → Shift carry → Repeat until carry is 0.
- Time Complexity: $O(\log(\max(a, b)))$
- Space Complexity: $O(1)$

15. Reverse a given number.

- Logic: Reverse the number by extracting digits and building the new number.

```

public class ReverseNumber {
    public static void main(String[] args) {
        int num = 12345;
        int reversed = 0;
        while (num > 0) {
            reversed = reversed * 10 + num % 10;
            num = num / 10;
        }
    }
}

```

```

    }
    System.out.println("Reversed number: " + reversed);
}
}

```

- Flowchart: Input → Extract last digit → Build reversed number → Divide number by 10 → Repeat.
- Time Complexity: $O(d)$
- Space Complexity: $O(1)$

16. Find GCD (Greatest Common Divisor) of two given numbers.

- Logic: Use Euclid's algorithm for GCD: $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$.

```

public class GCD {
    public static void main(String[] args) {
        int a = 56, b = 98;
        while (b != 0) {
            int temp = b;
            b = a % b;
            a = temp;
        }
        System.out.println("GCD: " + a);
    }
}

```

- Flowchart: Input → Use Euclid's algorithm → Output GCD.
- Time Complexity: $O(\log(\min(a, b)))$
- Space Complexity: $O(1)$

17. Find LCM (Least Common Multiple) of two given numbers.

- Logic: LCM can be found using the formula: $\text{LCM}(a, b) = (a * b) / \text{GCD}(a, b)$.

```

public class LCM {
    public static void main(String[] args) {
        int a = 15, b = 20;
        int gcd = 1;
        int num1 = a, num2 = b;

        while (num2 != 0) {
            int temp = num2;

```

```

        num2 = num1 % num2;
        num1 = temp;
    }

    gcd = num1;
    int lcm = (a * b) / gcd;
    System.out.println("LCM: " + lcm);
}
}

```

- Flowchart: Input → Find GCD → Calculate LCM → Output.
- Time Complexity: $O(\log(\min(a, b)))$
- Space Complexity: $O(1)$

18. Find LCM of two given numbers using the Prime Factors method.

- Logic: Find the prime factors of both numbers, then multiply the highest powers of all prime factors.

```

import java.util.*;

public class LCMPrimeFactors {
    public static void primeFactors(int n, Map<Integer, Integer>
factors) {
        for (int i = 2; i * i <= n; i++) {
            while (n % i == 0) {
                factors.put(i, factors.getOrDefault(i, 0) + 1);
                n /= i;
            }
        }
        if (n > 1) {
            factors.put(n, factors.getOrDefault(n, 0) + 1);
        }
    }

    public static void main(String[] args) {
        int a = 12, b = 15;
        Map<Integer, Integer> factorsA = new HashMap<>();
        Map<Integer, Integer> factorsB = new HashMap<>();

        primeFactors(a, factorsA);
        primeFactors(b, factorsB);
    }
}

```

```

    int lcm = 1;
    Set<Integer> primes = new HashSet<>(factorsA.keySet());
    primes.addAll(factorsB.keySet());

    for (int prime : primes) {
        int power = Math.max(factorsA.getOrDefault(prime, 0),
factorsB.getOrDefault(prime, 0));
        lcm *= Math.pow(prime, power);
    }

    System.out.println("LCM: " + lcm);
}
}

```

- Flowchart: Input → Find prime factors of both numbers → Multiply highest powers of primes → Output LCM.
- Time Complexity: $O(\sqrt{n})$ for each number.
- Space Complexity: $O(n)$ for storing factors.

19. Check whether the given number is a Palindrome or NOT.

- Logic: A number is a palindrome if it reads the same forward and backward.

```

public class Palindrome {
    public static void main(String[] args) {
        int num = 121, original = num, reversed = 0;
        while (num > 0) {
            reversed = reversed * 10 + num % 10;
            num /= 10;
        }
        if (original == reversed) {
            System.out.println(original + " is a Palindrome");
        } else {
            System.out.println(original + " is not a Palindrome");
        }
    }
}

```

- Flowchart: Input → Reverse number → Compare original and reversed → Output.
- Time Complexity: $O(d)$ where **d** is the number of digits.

- Space Complexity: $O(1)$

20. Print all the Prime Factors of the Given Number.

- Logic: To get prime factors, divide the number by every prime starting from 2, and repeat the process with the quotient.

```
public class PrimeFactors {
    public static void main(String[] args) {
        int num = 60;
        for (int i = 2; i * i <= num; i++) {
            while (num % i == 0) {
                System.out.println(i);
                num /= i;
            }
        }
        if (num > 1) {
            System.out.println(num);
        }
    }
}
```

- Flowchart: Input → Loop through primes → Divide the number by primes → Output prime factors.
- Time Complexity: $O(\sqrt{n})$
- Space Complexity: $O(1)$

21. Check whether the Given Number is Prime Number or NOT.

- Logic: A prime number is divisible only by 1 and itself.

```
public class PrimeNumber {
    public static void main(String[] args) {
        int num = 29;
        boolean isPrime = true;
        for (int i = 2; i <= Math.sqrt(num); i++) {
            if (num % i == 0) {
                isPrime = false;
                break;
            }
        }
        if (isPrime && num > 1) {
            System.out.println(num + " is Prime");
        }
    }
}
```

```

        } else {
            System.out.println(num + " is not Prime");
        }
    }
}

```

- Flowchart: Input → Loop from 2 to sqrt(num) → Check divisibility → Output.
- Time Complexity: $O(\sqrt{n})$
- Space Complexity: $O(1)$

22. Print Prime Numbers from 1 to N.

- Logic: Use the sieve method or check each number for primality.

```

public class PrimeNumbersUpToN {
    public static void main(String[] args) {
        int n = 30;
        for (int num = 2; num <= n; num++) {
            boolean isPrime = true;
            for (int i = 2; i <= Math.sqrt(num); i++) {
                if (num % i == 0) {
                    isPrime = false;
                    break;
                }
            }
            if (isPrime) {
                System.out.println(num);
            }
        }
    }
}

```

- Flowchart: Input N → Loop through numbers → Check if prime → Print prime numbers.
- Time Complexity: $O(N * \sqrt{N})$
- Space Complexity: $O(1)$

23. Check whether the given number is an Armstrong Number or NOT.

- Logic: A number is an Armstrong number if the sum of its digits raised to the power of the number of digits equals the number itself.

```

public class ArmstrongNumber {
    public static void main(String[] args) {
        int num = 153, original = num, sum = 0, digits =
String.valueOf(num).length();
        while (num > 0) {
            sum += Math.pow(num % 10, digits);
            num /= 10;
        }
        if (original == sum) {
            System.out.println(original + " is an Armstrong
number");
        } else {
            System.out.println(original + " is not an Armstrong
number");
        }
    }
}

```

- Flowchart: Input → Find number of digits → Raise each digit to the power of digits → Sum → Compare with original.
- Time Complexity: $O(d)$, where d is the number of digits.
- Space Complexity: $O(1)$

24. Print Armstrong Numbers between 1 to 1000.

- Logic: Iterate through numbers, check if each is an Armstrong number.

```

public class ArmstrongNumbersRange {
    public static void main(String[] args) {
        for (int num = 1; num <= 1000; num++) {
            int original = num, sum = 0, digits =
String.valueOf(num).length();
            while (num > 0) {
                sum += Math.pow(num % 10, digits);
                num /= 10;
            }
            if (original == sum) {
                System.out.println(original);
            }
        }
    }
}

```

- Flowchart: Input → Loop through numbers → Check for Armstrong number → Output Armstrong numbers.
- Time Complexity: $O(N * d)$, where **N** is the range and **d** is the number of digits.
- Space Complexity: $O(1)$

25. Check whether the given number is a Perfect Number or NOT.

- Logic: A perfect number is equal to the sum of its proper divisors (excluding itself).

```
public class PerfectNumber {
    public static void main(String[] args) {
        int num = 6, sum = 0;
        for (int i = 1; i <= num / 2; i++) {
            if (num % i == 0) {
                sum += i;
            }
        }
        if (num == sum) {
            System.out.println(num + " is a Perfect number");
        } else {
            System.out.println(num + " is not a Perfect number");
        }
    }
}
```

- Flowchart: Input → Loop through divisors → Sum divisors → Compare with original number → Output.
- Time Complexity: $O(n)$
- Space Complexity: $O(1)$

26. Print Perfect Numbers between 1 to 1000.

- Logic: Iterate through numbers and check if each is a perfect number.

```
public class PerfectNumbersRange {
    public static void main(String[] args) {
        for (int num = 1; num <= 1000; num++) {
            int sum = 0;
            for (int i = 1; i <= num / 2; i++) {
                if (num % i == 0) {
                    sum += i;
                }
            }
            if (num == sum) {
                System.out.println(num);
            }
        }
    }
}
```

```
        }
    }
    if (num == sum) {
        System.out.println(num);
    }
}
}
```

- **Flowchart:** Input → Loop through numbers → Check if perfect number → Output perfect numbers.
- **Time Complexity:** $O(N * n)$
- **Space Complexity:** $O(1)$